“Compiler Design

Unit 1

Overview of Compilation

Available on notes.arpanadhikari7.com.np

Introduction

- A compiler is a tool that translates software written in one language into another language.
- To translate text from one language to another, the tool must understand both the form, or syntax, and content, or meaning, of the input language.
- It needs to understand the rules that govern syntax and meaning in the output language.
- The structure of a typical compiler derives from these simple observations.
- The compiler has a front end to deal with the source language.
- It has a back end to deal with the target language.
- Connecting the front end and the back end, it has a formal structure for representing the program in an intermediate form whose meaning is largely independent of either language.

Available on notes.arpanadhikari7.com.np

Overview of Computer Programming

- Computer programs are simply sequences of abstract operations written in a *programming language—a formal language designed for expressing computation.*
- Programming languages have rigid properties and meanings.
- Programming languages are designed for expressiveness, conciseness, and clarity.
- Programming languages are designed to avoid ambiguity; an ambiguous program has no meaning.
- Programming languages are designed to specify computations—to record the sequence of actions that perform some task or produce some results.

Available on notes.arpanadhikari7.com.np

- Programming languages are, in general, designed to allow humans to express computations as sequences of operations.
- Computer processors, hereafter referred to as processors, microprocessors are designed to execute sequences of operations.
- The single programming language statement must be translated into literally hundreds of machine operations before it can execute.
- The tool that performs such translations is called a compiler.
- The compiler takes as input a program written in some language and produces as its output an equivalent program.
- In the classic notion of a compiler, the output program is expressed in the operations available on some specific processor, often called the target machine.



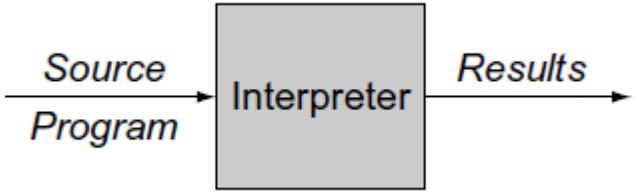
Available on notes.arpanadhikari7.com.np

- Some compilers produce a target program written in a human-oriented programming language rather than the assembly language of some computer.
- The programs that these compilers produce require further translation before they can execute directly on a computer.
- Compiler is a program which translates source program written in one language to an equivalent program in other language (the target language).
- Usually the source language is a high level language like Java, C, C++ etc. whereas the target language is machine code or "code" that a computer's processor understands.
- Many modern compilers have a common 'two stage' design.
- The "front end" translates the source language or the high level program into an intermediate representation. The second stage is the "back end", which works with the internal representation to produce low level code.

Available on notes.arpanadhikari7.com.np

Interpreter

- Many other systems qualify as compilers. For example, a typesetting program that produces PostScript can be considered a compiler.
- It takes as input a specification for how the document should look on the printed page and it produces as output a PostScript file.
- PostScript is simply a language for describing images.
- Because the typesetting program takes an executable specification and produces another executable specification, it is a compiler.
- The code that turns PostScript into pixels is typically an *interpreter, not* a compiler.
- An interpreter takes as input an executable specification and produces as output the result of executing the specification.



Available on notes.arpanadhikari7.com.np

- Some languages, such as Perl, Scheme, and APL, are more often implemented with interpreters than with compilers.
- Some languages adopt translation schemes that include both compilation and interpretation.
- Java is compiled from source code into a form called *bytecode*.
- Java applications execute by running the bytecode on the corresponding Java Virtual Machine (jvm), an interpreter for bytecode.
- Simply an interpreter is a program that reads and executes code directly, line by line or instruction by instruction, without converting it into machine code beforehand (like a compiler does).

Available on notes.arpanadhikari7.com.np

- Interpreters and compilers have much in common. They perform many of the same tasks.
- Both analyze the input program and determine whether or not it is a valid program.
- Both build an internal model of the structure and meaning of the program.
- Both determine where to store values during execution.
- However, interpreting the code to produce a result is quite different from emitting a translated program that can be executed to produce the result.

Available on notes.arpanadhikari7.com.np

COMPILER	INTERPRETER
Compiler works on the complete program at once. It takes the entire program as input.	Interpreter program works line-by-line. It takes one statement at a time as input.
Compiler generates intermediate code, called the object code or machine code.	Interpreter does not generate intermediate object code or machine code.
Compiler executes conditional control statements (like if-else and switch-case) and logical constructs faster than interpreter.	Interpreter execute conditional control statements at a much slower speed.
Compiled programs take more memory because the entire object code has to reside in memory.	Interpreter does not generate intermediate object code. As a result, interpreted programs are more memory efficient.
Compile once and run anytime. Compiled program does not need to be compiled every time.	Interpreted programs are interpreted line-by-line every time they are run.
Errors are reported after the entire program is checked for syntactical and other errors.	Error is reported as soon as the first error is encountered. Rest of the program will not be checked until the existing error is removed.
A compiled language is more difficult to debug.	Debugging is easy because interpreter stops and reports errors as it encounters them.

Available on notes.arpanadhikari7.com.np

Compiler does not allow a program to run until it is completely error-free.	**Interpreter runs the program from first line and stops execution only if it encounters an error.**
Compiled languages are more efficient but difficult to debug.	Interpreted languages are less efficient but easier to debug. This makes such languages an ideal choice for new students.
Examples of programming languages that use compilers: C, C++, COBOL	**Examples** of programming languages that use interpreters: BASIC, Visual Basic, Python, Ruby, PHP, Perl, MATLAB, Lisp

Available on notes.arpanadhikari7.com.np

Advantages Of Compiler

- The program codes are translated into machine code already and hence the time to execute the code is very less.
- EXE files execute faster than source codes. The object codes are saved and can run at any time.
- Object Codes are harder for a user to change than source codes
- The compiler checks the source code for syntax errors during compilation.

Disadvantages of Compiler

- As the entire program is translated it uses much more of the computer's memory.
- It is not possible to change the program without going back to the source code while working with a compiler.
- Object code needs to be produced before a final executable file, this can be a slow process.
- The Source code must be 100% correct for the executable file to be produced.

Available on notes.arpanadhikari7.com.np

Advantages Of Interpreter

- Interpreters make working with the source code much easier.
- It translates one instruction at a time; therefore it uses a minimum amount of memory.
- It is also helpful in the debugging process because the interpreter can relate error messages to the instruction being executed

Disadvantages of Interpreter

- Each time the program is executed time is spent interpreting, hence a slower execution time.

Available on notes.arpanadhikari7.com.np

Why study compiler

- A good compiler contains a microcosm of computer science.
- It makes practical use of greedy algorithms (register allocation), heuristic search techniques (list scheduling), graph algorithms (dead-code elimination), dynamic programming (instruction selection), finite automata and push-down automata (scanning and parsing), and fixed-point algorithms (data-flow analysis).
- It deals with problems such as dynamic allocation, synchronization, naming, locality, memory hierarchy management, and pipeline scheduling.

Available on notes.arpanadhikari7.com.np

- Compilers play a fundamental role in the central activity of computer science: preparing problems for solution by computer.
- Most software is compiled, and the correctness of that process and the efficiency of the resulting code have a direct impact on our ability to build large systems.
- Compiler uses the scanner and parsers the same tools are used for text searching, website filtering, word processing, and command-language interpreters.
- Code generators use algorithms for tree-pattern matching, parsing, dynamic programming, and text matching to automate the selection of instructions.

Available on notes.arpanadhikari7.com.np

The Fundamental Principles of Compilation

- Compilers are large, complex, carefully engineered objects.
- While many issues in compiler design are amenable to multiple solutions and interpretations, there are two fundamental principles that a compiler writer must keep in mind at all times. The first principle is inviolable:

The compiler must preserve the meaning of the program being compiled.

- Correctness is a fundamental issue in programming.
- The compiler must preserve correctness by faithfully implementing the "meaning" of its input program.

Available on notes.arpanadhikari7.com.np

- The second principle that a compiler must observe is

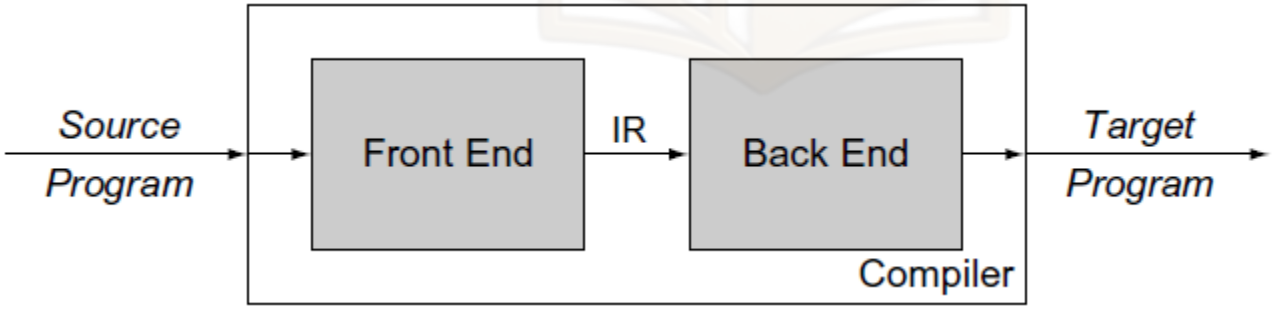
The compiler must improve the input program in some discernible way.

- A traditional compiler improves the input program by making it directly executable on some target machine.
- Other "compilers" improve their input in different ways.
- For example, tpic is a program that takes the specification for a drawing written in the graphics language pic and converts it into LATEX; the "improvement" lies in LATEX's greater availability and generality.

Available on notes.arpanadhikari7.com.np

Compiler Structure

- As the single-box model suggests, a compiler must both understand the source program that it takes as input and map its functionality to the target machine.
- The distinct nature of these two tasks suggests a division of labor -and leads to a design that decomposes compilation into two major pieces: a front end and back end.



- A compiler uses some set of data structures to represent the code that it processes. That form is called an *intermediate representation, or IR.*

Available on notes.arpanadhikari7.com.np

- The front end focuses on understanding the source-language program.
- The back end focuses on mapping programs to the target machine.
- The front end must encode its knowledge of the source program in some **IR structure for later use by the back end.**
- **This *intermediate representation (IR)*** becomes the compiler's definitive representation for the code it is translating.
- At each point in compilation, the compiler will have a definitive representation.
- It may, in fact, use several different IRs as compilation progresses, but at each point, one representation will be the definitive IR.
- A **definitive IR** refers to a well-defined, standardized, and optimized representation that serves as a central data structure for compiler transformations.

Available on notes.arpanadhikari7.com.np

- In a two-phase compiler, the front end must ensure that the source program is well formed, and it must map that code into the IR.
- The back end must map the IR program into the instruction set and the finite resources of the target machine.
- Because the back end only processes IR created by the front end, it can assume that the IR contains no syntactic or semantic errors.
- The compiler can make multiple passes over the IR form of the code before emitting the target program.
- This should lead to better code, as the compiler can, in effect, study the code in one phase and record relevant details.
- Then, in later phases, it can use these recorded facts to improve the quality of translation.
- This strategy requires that knowledge derived in the first pass be recorded in the IR, where later passes can find and use it.

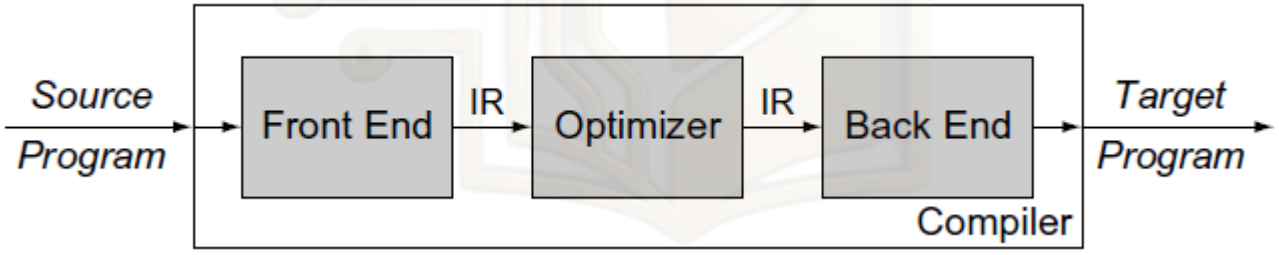
Available on notes.arpanadhikari7.com.np

- Finally, the two-phase structure may simplify the process of *retargeting* the compiler.
- The task of changing the compiler to generate code for a new processor is often called *retargeting the compiler*.
- We can easily envision constructing multiple back ends for a single front end to produce compilers that accept the same language but target different machines.
- Similarly, we can envision front ends for different languages producing the same IR and using a common back end.

Available on notes.arpanadhikari7.com.np

- Introducing an IR makes it possible to add more phases to compilation.
- The compiler writer can insert a third phase between the front end and the back end.
- This middle section, or *optimizer, takes an IR program as its input and* produces a semantically equivalent IR program as its output.
- By using the IR as an interface, the compiler writer can insert this third phase with minimal disruption to the front end and back end.
- This leads to the following compiler structure, termed a *three-phase compiler*.

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

- The optimizer is an IR-to-IR transformer that tries to improve the IR program in some way.
- The optimizer can make one or more passes over the IR, analyze the IR, and rewrite the IR.
- The optimizer may rewrite the IR in a way that is likely to produce a faster target program from the back end or a smaller target program from the back end.

Available on notes.arpanadhikari7.com.np

- Conceptually, the three-phase structure represents the classic optimizing compiler.
- In practice, each phase is divided internally into a series of passes.
- The front end consists of two or three passes that handle the details of recognizing valid source-language programs and producing the initial IR form of the program.
- The middle section contains passes that perform different optimizations.
- The number and purpose of these passes vary from compiler to compiler.
- The back end consists of a series of passes, each of which takes the IR program one step closer to the target machine's instruction set.

Available on notes.arpanadhikari7.com.np

- In practice, the conceptual division of a compiler into three phases, a front end, a middle section or optimizer, and a back end, is useful.
- The problems addressed by these phases are different.
- The front end is concerned with understanding the source program and recording the results of its analysis into IR form.
- The optimizer section focuses on improving the IR form.
- The back end must map the transformed IR program onto the bounded resources of the target machine in a way that leads to efficient use of those resources.

Available on notes.arpanadhikari7.com.np

- Of these three phases, the optimizer has the murkiest description.
- The term optimization implies that the compiler discovers an optimal solution to some problem.
- The issues and problems that arise in optimization are so complex and so interrelated that they cannot, in practice, be solved optimally.
- Furthermore, the actual behavior of the compiled code depends on interactions among all of the techniques applied in the optimizer and the back end.
- Thus, even if a single technique can be proved optimal, its interactions with other techniques may produce less than optimal results.
- As a result, a good optimizing compiler can improve the quality of the code, relative to an un optimized version.
- However, an optimizing compiler will almost always fail to produce optimal code.
- The choice between these two approaches depends on the constraints under which the compiler is built and operates.

Available on notes.arpanadhikari7.com.np

- The compilation process is divided into three stages: the front end, the middle end, and the back end.
- The front end verifies syntax and semantics according to a specific source language, while the middle end performs optimizations on the IR that are independent of the CPU architecture being targeted.
- The back end generates target-dependent assembly code and performs analysis, transformations, and optimizations that are specific to the target CPU architecture.
- The front/middle/back-end approach makes it possible to combine front ends for different languages with back ends for different CPUs while sharing the optimizations of the middle end.

Available on notes.arpanadhikari7.com.np

The Front End

- Before the compiler can translate an expression into executable target machine code, it must understand both its form, or *syntax, and its meaning,* or *semantics*.
- This stage of compilation aims to detect any programmatic errors with the source code.
- It does this by performing lexical analysis, parsing (or syntax analysis) and semantic analysis.
- The output of the front end is an intermediate representation of the code, which can be passed to the middle end.

Available on notes.arpanadhikari7.com.np

- ***Checking Syntax***
- To check the syntax of the input program, the compiler must compare the program's structure against a definition for the language.
- This requires an appropriate formal definition, an efficient mechanism for testing whether or not the input meets that definition, and a plan for how to proceed on an illegal input.
- Mathematically, the source language is a set, usually infinite, of strings defined by some finite set of rules, called a *grammar*.
- *Two separate passes* in the front end, called the scanner and the parser, determine whether or not the input code is, in fact, a member of the set of valid programs defined by the grammar.

Available on notes.arpanadhikari7.com.np

- The scanner (lexer) and parser are fundamental components of a compiler's frontend.
- They work together to analyze the structure of the source code before semantic analysis and code generation.
- The scanner is responsible for lexical analysis, breaking the source code into meaningful tokens (keywords, identifiers, operators, etc.)
- The scanner also checks for lexical errors (e.g., invalid characters, unmatched strings).
- The parser checks if the sequence of tokens follows the grammar rules of the language.
- Parser constructs an Abstract Syntax Tree (AST) if the syntax is correct.
- An AST is a simplified version of the parse tree, retaining only the essential structure needed for semantic analysis and code generation.
- Parser reports syntax errors (e.g., missing semicolons, mismatched braces).

Available on notes.arpanadhikari7.com.np

Intermediate Representations

- The final issue handled in the front end of a compiler is the generation of an IR form of the code.
- Compilers use a variety of different kinds of IR, depending on the source language, the target language, and the specific transformations that the compiler applies.
- Some IRS represent the program as a graph.
- Others resemble a sequential assembly code program.
- A **low-level sequential IR** is a machine-like representation of code used in compilers after high-level optimizations but before final code generation.
- **low-level sequential IR** bridges the gap between architecture-independent optimizations and backend-specific machine code.
- The following example expression might look in a low-level, sequential IR

Available on notes.arpanadhikari7.com.np

$$a \leftarrow a \times 2 \times b \times c \times d$$

$$t_0 \leftarrow a \times 2$$

$$t_1 \leftarrow t_0 \times b$$

$$t_2 \leftarrow t_1 \times c$$

$$t_3 \leftarrow t_2 \times d$$

$$a \leftarrow t_3$$

Available on notes.arpanadhikari7.com.np

Optimizer

- The **optimizer** is a critical component of a compiler that improves the intermediate representation (IR) of code to enhance performance, reduce size, or meet other objectives while preserving program semantics.
- Optimization occurs primarily on the IR but can also happen at later stages (e.g., machine code).
- The optimizer analyzes the IR form of the code to discover facts about that context and uses that contextual knowledge to rewrite the code so that it computes the same answer in a more efficient way.
- Efficiency can have many meanings.
- The classic notion of optimization is to reduce the application's running time.
- In other contexts, the optimizer might try to reduce the size of the compiled code, or other properties such as the energy that the processor consumes evaluating the code. All of these strategies target efficiency.

Available on notes.arpanadhikari7.com.np

```
b ← ...
c ← ...
a ← 1
for i = 1 to n
  read d
  a ← a × 2 × b × c × d
  end
```

(a) Original Code in Context

```
b ← ...
c ← ...
a ← 1
t ← 2 × b × c
for i = 1 to n
  read d
  a ← a × d × t
  end
```

(b) Improved Code

Available on notes.arpanadhikari7.com.np

Analysis

- Most optimizations consist of an analysis and a transformation.
- The analysis determines where the compiler can safely and profitably apply the technique.
- Compilers use several kinds of analysis to support transformations.
- Data flow analysis reasons, at compile time, about the flow of values at runtime.
- Data-flow analyzers typically solve a system of simultaneous set equations that are derived from the structure of the code being translated.
- Dependence analysis uses number-theoretic tests to reason about the values that can be assumed by subscript expressions.
- It is used to disambiguate references to array elements. .

Available on notes.arpanadhikari7.com.np

Transformation

- To improve the code, the compiler must go beyond analyzing it.
- The compiler must use the results of analysis to rewrite the code into a more efficient form.
- Myriad transformations have been invented to improve the time or space requirements of executable code.
- Some, such as discovering loop-invariant computations and moving them to less frequently executed locations, improve the running time of the program.
- Transformations vary in their effect, the scope over which they operate, and the analysis required to support them.

Available on notes.arpanadhikari7.com.np

The Back End

- The compiler's back end traverses the IR form of the code and emits code for the target machine.
- It selects target-machine operations to implement each IR operation.
- It chooses an order in which the operations will execute efficiently.
- It decides which values will reside in registers and which values will reside in memory and inserts code to enforce those decisions.

Available on notes.arpanadhikari7.com.np

Instruction Selection

- The first stage of code generation rewrites the IR operations into target machine operations, a process called instruction selection.
- Instruction selection maps each IR operation, in its context, into one or more target machine operations.
- Consider the example expression, a = a × 2 × b × c × d, into code for the ILOC ("intermediate language for an optimizing compiler".) to illustrate the process.
- This code assumes that a, b, c, and d are located at offsets @a, @b, @c, and @d from an address contained in the register r_{arp}.

Available on notes.arpanadhikari7.com.np

```
loadAI   r_arp, @a  ⇒ r_a      // load 'a'
loadI    2          ⇒ r_2      // constant 2 into r_2
loadAI   r_arp, @b  ⇒ r_b      // load 'b'
loadAI   r_arp, @c  ⇒ r_c      // load 'c'
loadAI   r_arp, @d  ⇒ r_d      // load 'd'
mult     r_a, r_2   ⇒ r_a      // r_a ← a × 2
mult     r_a, r_b   ⇒ r_a      // r_a ← (a × 2) × b
mult     r_a, r_c   ⇒ r_a      // r_a ← (a × 2 × b) × c
mult     r_a, r_d   ⇒ r_a      // r_a ← (a × 2 × b × c) × d
storeAI  r_a        ⇒ r_arp,@a // write r_a back to 'a'
```

Figure: ILOC Code for a a × 2 × b × c × d

Available on notes.arpanadhikari7.com.np

- The compiler has chosen a straightforward sequence of operations.
- It loads all of the relevant values into registers, performs the multiplications in order, and stores the result to the memory location for a.
- It assumes an unlimited supply of registers and names them with symbolic names such as r_a to hold a and r_{arp} to hold the address where the data storage for our named values begins.

Available on notes.arpanadhikari7.com.np

- The instruction selector can take advantage of special operations on the target machine.
- For example, if an immediate-multiply operation (multI) is available, it might replace the operation mult r_a, r_2 $\Rightarrow r_a$ with multI r_a, 2$\Rightarrow$ r_a, eliminating the need for the operation loadI 2$\Rightarrow r_2$ and reducing the demand for registers.
- If addition is faster than multiplication, it might replace mult r_a, $r_2 \Rightarrow r_a$ with add r_a, $r_a \Rightarrow r_a$, avoiding both the loadI and its use of r_2, as well as replacing the mult with a faster add.

Available on notes.arpanadhikari7.com.np

Register Allocation

- During instruction selection, the compiler deliberately ignored the fact that the target machine has a limited set of registers.
- Instead, it used virtual registers and assumed that "enough" registers existed.
- In practice, the earlier stages of compilation may create more demand for registers than the hardware can support.
- The register allocator must map those virtual registers onto actual target-machine registers.
- Thus, the register allocator decides, at each point in the code, which values should reside in the target-machine registers.
- It then rewrites the code to reflect its decisions.
- For example, a register allocator might minimize register use by rewriting the code from above example as:

Available on notes.arpanadhikari7.com.np

```
loadAI  r_arp, @a ⇒ r1        // load 'a'
add     r1, r1    ⇒ r1        // r1 ← a × 2
loadAI  r_arp, @b ⇒ r2        // load 'b'
mult    r1, r2    ⇒ r1        // r1 ← (a × 2) × b
loadAI  r_arp, @c ⇒ r2        // load 'c'
mult    r1, r2    ⇒ r1        // r1 ← (a × 2 × b) × c
loadAI  r_arp, @d ⇒ r2        // load 'd'
mult    r1, r2    ⇒ r1        // r1 ← (a × 2 × b × c) × d
storeAI r1        ⇒ r_arp, @a // write r_a back to 'a'
```

Available on notes.arpanadhikari7.com.np

- This sequence uses three registers instead of six.
- Minimizing register use may be counterproductive.
- If, for example, any of the named values, a, b, c, or d, are already in registers, the code should reference those registers directly.
- If all are in registers, the sequence could be implemented so that it required no additional registers.
- Alternatively, if some nearby expression also computed a × 2, it might be better to preserve that value in a register than to re compute it later.
- This optimization would increase demand for registers but eliminate a later instruction.

Available on notes.arpanadhikari7.com.np

Instruction Scheduling

- To produce code that executes quickly, the code generator may need to reorder operations to reflect the target machine's specific performance constraints.
- The execution time of the different operations can vary.
- Memory access operations can take tens or hundreds of cycles, while some arithmetic operations, particularly division, take several cycles.
- The impact of these longer latency operations on the performance of compiled code can be dramatic.
- Assume, for the moment, that a loadAI or storeAI operation requires three cycles, a mult requires two cycles, and all other operations require one cycle.
- The following table shows how the previous code fragment performs under these assumptions.
- The **Start column shows the cycle in which each operation** begins execution and the **End column shows the cycle in which it** Completes.

Available on notes.arpanadhikari7.com.np

Start	End				
1	3	loadAI	r_{arp}, @a	⇒ r_1	// load 'a'
4	4	add	r_1, r_1	⇒ r_1	// $r_1 \leftarrow a \times 2$
5	7	loadAI	r_{arp}, @b	⇒ r_2	// load 'b'
8	9	mult	r_1, r_2	⇒ r_1	// $r_1 \leftarrow (a \times 2) \times b$
10	12	loadAI	r_{arp}, @c	⇒ r_2	// load 'c'
13	14	mult	r_1, r_2	⇒ r_1	// $r_1 \leftarrow (a \times 2 \times b) \times c$
15	17	loadAI	r_{arp}, @d	⇒ r_2	// load 'd'
18	19	mult	r_1, r_2	⇒ r_1	// $r_1 \leftarrow (a \times 2 \times b \times c) \times d$
20	22	storeAI	r_1	⇒ r_{arp}, @a	// write r_a back to 'a'

Available on notes.arpanadhikari7.com.np

- This nine-operation sequence takes 22 cycles to execute. Minimizing register use did not lead to rapid execution.
- Many processors have a property by which they can initiate new operations while a long-latency operation executes.
- As long as the results of a long latency operation are not referenced until the operation completes, execution proceeds normally.
- If, however, some intervening operation tries to read the result of the long-latency operation prematurely, the processor delays the operation that needs the value until the long-latency operation completes.
- An operation cannot begin to execute until its operands are ready, and its results are not ready until the operation terminates.

Available on notes.arpanadhikari7.com.np

- The instruction scheduler reorders the operations in the code.
- It attempts to minimize the number of cycles wasted waiting for operands.
- Of course, the scheduler must ensure that the new sequence produces the same result as the original.
- In many cases, the scheduler can drastically improve the performance of "naive" code.
- For our example, a good scheduler might produce the following sequence:

Available on notes.arpanadhikari7.com.np

Start	End			
1	3	loadAI	r_{arp}, @a ⇒ r_1	// load 'a'
2	4	loadAI	r_{arp}, @b ⇒ r_2	// load 'b'
3	5	loadAI	r_{arp}, @c ⇒ r_3	// load 'c'
4	4	add	r_1, r_1 ⇒ r_1	// r_1 ← a × 2
5	6	mult	r_1, r_2 ⇒ r_1	// r_1 ← (a × 2) × b
6	8	loadAI	r_{arp}, @d ⇒ r_2	// load 'd'
7	8	mult	r_1, r_3 ⇒ r_1	// r_1 ← (a × 2 × b) × c
9	10	mult	r_1, r_2 ⇒ r_1	// r_1 ← (a × 2 × b × c) × d
11	13	storeAI	r_1 ⇒ r_{arp}, @a	// write r_a back to 'a'

This version of the code requires just 13 cycles to execute.
The code uses one more register than the minimal number.
It starts an operation in every cycle except 8, 10, and 12.
Other equivalent schedules are possible, as are equal-length schedules that use more registers.

Available on notes.arpanadhikari7.com.np